COURSE NAME: ***SOFTWARE ENGINEERING AND PROJECT MANAGEMENT***

COURSE CODE: BCS501

Module 1 - Software and Software Engineering and Process Models

The Nature of Software

Software Engineering is the product of two words — software and engineering. Software is more than just program code: a program is executable code serving a computational purpose, while software is a collection of executable code, associated libraries, and documentation. Software built for a specific requirement is called a software product.

Engineering is about developing products using well-defined, scientific principles and methods. Software Engineering is therefore the engineering branch associated with developing software products using well-defined scientific principles, methods, and procedures; its outcome is an efficient and reliable software product.

- **IEEE definition —** (1) the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software — i.e., the application of engineering to software, and (2) the study of approaches as in (1).
- **Fritz Bauer's definition —** software engineering is the establishment and use of sound engineering principles to obtain economically software that is reliable and works efficiently on real machines.

Software takes on a dual role — it is a Product and, at the same time, a Vehicle (Process) for delivering a product.

- **As a Product —** it produces, manages, modifies, acquires, and displays information. It is an information transformer, ranging from a single bit to a complex multimedia presentation drawn from many independent sources.
- **As a Vehicle (Process) —** software is the basis for control of other computers (operating systems), communication of information (networks), and creation/control of other programs (software tools and environments).

1.1.1 Defining Software

Software is defined by three elements:

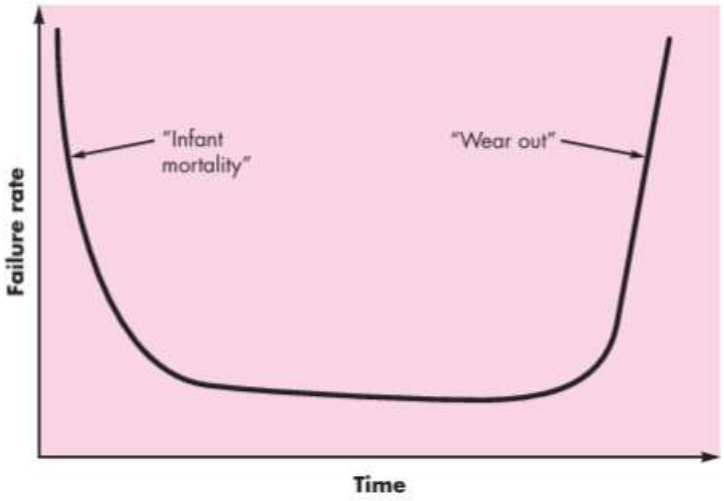
- Instructions — programs that, when executed, provide the desired function, features, and performance.

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

- Data structures — enable programs to adequately manipulate information.
- Documents — descriptive information, in hard-copy and virtual forms, describing the operation and use of the programs.

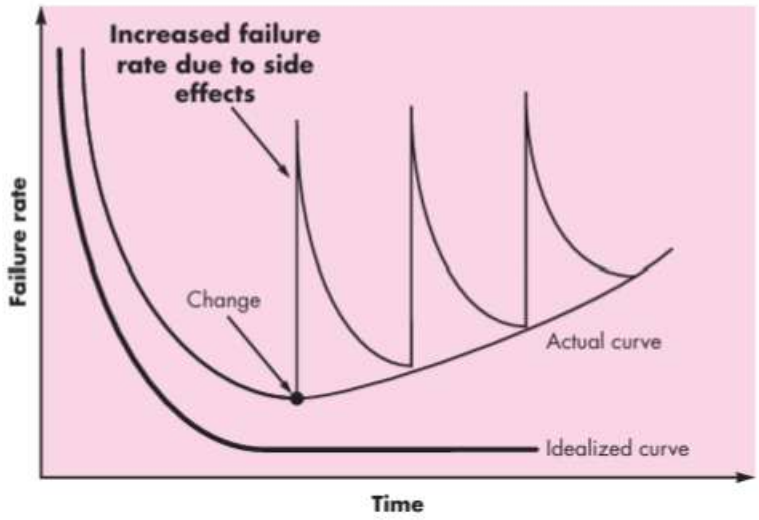
Characteristics of Software (how it differs from hardware)

- **1. Software is developed/engineered — it is not manufactured in the classical sense —** the two activities are fundamentally different: in manufacturing, the phase itself can introduce quality problems that don't exist for software; both depend on people, but the relationship differs; software costs are concentrated in engineering, so software projects cannot be managed like manufacturing.
- **2. Software doesn't "wear out" —** hardware exhibits high early failure rates (infant mortality), a flat useful-life region, then a rising wear-out curve as components ages.



Software has no such wear-out phase — there are no spare parts, and every failure indicates a design or coding error. However, software does deteriorate as frequent changes accumulate, giving an idealized failure curve with spikes at each change and a slowly increasing actual-failure curve.

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106



3. Most software is custom-built rather than assembled from components — although the industry is moving toward libraries of reusable components (e.g., GUI elements such as message windows and pull-down menus), in the hardware world component reuse is already a natural part of the engineering process.

1.1.2 Software Application Domains — seven broad categories

- **System software —** programs that service other programs (compilers, editors, file-management utilities, OS components, drivers, networking/telecom software); heavy hardware interaction, multi-user usage, scheduling and resource sharing.
- **Application software —** stand-alone programs solving a specific business need, often controlling business functions in real time (e.g., point-of-sale processing, manufacturing process control).
- **Engineering/scientific software —** characterized by "number-crunching" algorithms; astronomy, volcanology, automotive stress analysis, orbital dynamics, molecular biology, CAD, system simulation.
- **Embedded software —** resides within a product/system to implement and control features for the end user and the system itself (e.g., microwave keypad control, automotive fuel control, braking systems).
- **Product-line software —** provides a specific capability for many different customers — a limited/esoteric marketplace (inventory control) or a mass market (word processors, spreadsheets, graphics, entertainment, DBMS, financial apps).
- **Web applications ("WebApps") —** a network-centric software category, ranging from simple linked hypertext files to sophisticated computing tools integrated with corporate databases and business applications.

- **Artificial-intelligence software —** uses non-numerical algorithms to solve complex problems not amenable to computation/direct analysis; robotics, expert systems, pattern recognition, neural networks, theorem proving, game playing.

New Software Challenges

- **Open-world computing —** creating software that lets machines of all sizes communicate across vast networks (distributed, wireless computing).
- **Net sourcing —** architecting applications that benefit targeted end-user markets worldwide, treating the Web as a computing engine.
- **Open source —** distributing source code so customers can make local modifications easily and reliably.

1.1.3 Legacy Software

Legacy software refers to older programs developed decades ago. Its quality is often poor — inextensible design, convoluted code, and poor or non-existent documentation and test results. Legacy systems continue to evolve because the software must be: adapted to a new computing environment/technology; enhanced for new business requirements; extended to interoperate with modern systems/databases; and re-architected to remain viable within a network environment.

1.2 The Unique Nature of WebApps

In the early Web (1990–95), websites were little more than linked hypertext files (HTML) presenting text and limited graphics. As HTML was augmented by tools such as XML and Java, Web engineers began providing computing capability alongside informational content, giving rise to WebApps — sophisticated computing tools integrated with corporate databases and business applications, blending print publishing with software development, marketing with computing, internal communication with external relations, and art with technology.

Attributes found in the vast majority of WebApps:

- **Network intensiveness —** a WebApp resides on a network, serving a diverse client community, with access ranging from worldwide (Internet) to limited (corporate Intranet).
- **Concurrency —** a large number of users may access the WebApp at once, with widely varying usage patterns.
- **Unpredictable load —** the number of users can vary by orders of magnitude from day to day.
- **Performance —** must work effectively in terms of processing speed, or users go

elsewhere.

- **Availability —** 100% availability is unreasonable, but popular WebApps are often expected to be available 24/7/365.
- **Data driven —** primarily uses hypermedia to present text, graphics, audio, and video, often accessing databases outside the Web environment (e.g., e-commerce, financial apps).
- **Content sensitive —** the quality and aesthetic nature of content is an important determinant of quality.
- **Continuous evolution**

unlike conventional software with planned, chronologically spaced releases, WebApps evolve continuously.

- **Immediacy —** a compelling need to get software to market quickly, often within days or weeks.
- **Security —** because WebApps are accessible via a network, it is hard to limit who can access them, so strong protection of sensitive content is required.
- **Aesthetics —** look and feel is a key part of a WebApp's appeal, especially where it markets or sells products/ideas.

1.3 Software Engineering — A Layered Technology

To build software ready to meet modern challenges, a few simple realities must be recognized: problems should be understood before a solution is developed; design is a pivotal activity; software should exhibit high quality; and software should be maintainable. These realities lead to one conclusion — software, in all its forms and across all application domains, should be engineered.

Software Engineering is a fully layered technology — each layer rests on, and demands the fulfilment of, the layer below it:



Fig 1.3 — Software Engineering as a layered technology: Tools rest on Methods, which rest on a Process, which itself rests on an organizational commitment to quality (A Quality Focus).

- **A quality focus —** the bedrock that supports software engineering; any engineering approach must rest on an organizational commitment to quality.
- **Process —** the foundation layer; the software engineering process is the glue holding the technology layers together, enabling rational and timely development, and defining the framework for effective delivery of technology.
- **Methods —** provide the technical "how-to's" for building software, covering communication, requirements analysis, design modelling, program construction, testing, and support.
- **Tools —** provide automated or semi-automated support for the process and methods; when tools are integrated so information from one feeds another, the result is Computer-Aided Software Engineering (CASE).

1.4 The Software Process

A process is a collection of activities, actions, and tasks performed when some work product is to be created.

- **Activity —** strives to achieve a broad objective (e.g., communication with stakeholders), applied regardless of application domain, project size, effort complexity, or rigor.
- **Action —** encompasses a set of tasks that produce a major work product (e.g., an architectural design model).
- **Task —** focuses on a small, well-defined objective (e.g., conducting a unit test)

producing a tangible outcome.

A process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities applicable to all software projects, plus a set of umbrella activities applicable across the entire process.

Five generic framework activities

- **Communication —** collaborate with the customer to understand stakeholders' objectives and gather requirements defining software features and functions.
- **Planning —** creates a "software project plan" — a map describing the technical tasks, likely risks, required resources, work products, and work schedule.
- **Modeling —** creation of models to help developers and customers understand requirements and software design.
- **Construction —** combines code generation and the testing needed to uncover errors in the code.
- **Deployment —** the software is delivered to the customer, who evaluates it and provides feedback.

These five activities apply equally to small, simple programs, large Web applications, and large, complex computer-based systems.

Umbrella activities

- Software project tracking and control — assess progress against the plan and act to maintain the schedule.
- Risk management — assess risks that may affect project outcome or product quality.
- Software quality assurance — define and conduct activities that ensure software quality.
- Technical reviews — assess work products to uncover and remove errors before they propagate.
- Measurement — define and collect process, project, and product measures to help meet stakeholder needs.
- Software configuration management — manage the effects of change throughout the process.
- Reusability management — define criteria for work-product reuse and establish reuse mechanisms.
- Work product preparation and production — create work products such as models, documents, logs, forms, and lists.

The software process is not rigid — it must be agile and adaptable, so the process adopted for one project may differ significantly from another. Differences include:

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

overall flow and interdependency among tasks; degree to which tasks are defined within each activity; degree to which work products are identified/required; how QA, tracking, and control activities are applied; overall detail and rigor of the process description; degree of stakeholder involvement; level of team autonomy; and how strictly team organization/roles are prescribed.

1.5 The Software Engineering Practice

1.5.1 The Essence of Practice

Four essential steps, regardless of the process model used:

- Understand the problem (communication and analysis).
- Plan a solution (software design).
- Carry out the plan (code generation).
- Examine the result for accuracy (testing and quality assurance).

Each step raises a series of guiding questions — Understand the problem: Who are the stakeholders? What are the unknowns? Can the problem be decomposed into smaller ones? Can a graphic analysis model be created? Plan the solution: Have similar problems been solved before? Can sub-problems be defined? Can a design model be created? Carry out the plan: Does the solution conform to the plan? Is code traceable to the design? Has design/code been reviewed? Examine the result: Can each component be tested? Does the solution meet the required data, functions, and features?

1.5.2 Software General Principles — David Hooker's seven principles

- **1. The Reason It All Exists —** a software system exists for one reason: to provide value to its users.
- **2. KISS (Keep It Simple, Stupid!) —** all design should be as simple as possible, but no simpler.
- **3. Maintain the Vision —** a clear vision is essential to a project's success; without one it ends up “of two or more minds” about itself.
- **4. What You Produce, Others Will Consume —** always specify, design, and implement knowing someone else will have to understand your work.
- **5. Be Open to the Future —** a system with a long lifetime has more value; never design yourself into a corner.
- **6. Plan Ahead for Reuse —** planning ahead for reuse reduces cost and increases the value of both the reusable components and the systems that use them.
- **7. Think! —** placing clear, complete thought before action almost always produces better results.

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

1.6 Software Myths

Software myths are beliefs about software and its development process, traceable to the earliest days of computing. They appear to be reasonable statements of fact, have an intuitive feel, and are often promulgated by experienced practitioners who "know the score" — which is exactly what makes them dangerous.

Management Myths

Myth	Reality
We already have a book full of standards and procedures — won't that give my people everything they need?	The book may exist, but is it used, known, modern, complete, adaptable, and streamlined for quality? In many cases, the answer is NO.
If we fall behind schedule, we can add more programmers and catch up.	"Adding people to a late project makes it later" — new people need training time from existing staff, reducing productive effort. People can be added, but only in a planned, coordinated manner.
If we outsource the project to a third party, I can just relax and let them build it.	An organization that cannot manage/control software projects internally will invariably struggle when it outsources them.

Customer Myths

Myth	Reality
A general statement of objectives is enough to start writing programs — we can fill in details later.	An ambiguous statement of objectives is a recipe for disaster. Unambiguous requirements come only from continuous communication between customer and developer.
Project requirements keep changing, but that's fine — software is flexible.	Requirements do change, but the cost impact grows rapidly with time — early change is cheap; late

	change forces upheaval in committed resources and design.

Practitioner's Myths

Myth	Reality
Once we write the program and get it working, our job is done.	60–80% of all effort on software is consumed after it is first delivered to the customer.
Until I get the program running, I have no way of assessing its quality.	Formal technical reviews are an effective quality-assurance mechanism from project inception, and are often more effective than testing at finding certain classes of defects.
The only deliverable work product for a successful project is the working program.	A working program is only one part of the software configuration — documentation provides a foundation for engineering and support.
Software engineering will make us create voluminous, unnecessary documentation and slow us down.	Software engineering is about creating quality, not documents. Better quality reduces rework, and reduced rework means faster delivery.

Chapter 2 — Process Models

2.1 A Generic Process Model

A process is a collection of work activities, actions, and tasks performed when a work product is to be created; these reside within a framework or model that defines their relationship with the process and with one another. Each framework activity is populated by a set of software engineering actions, and each action is defined by a task set that identifies the work tasks to be completed, the work products to be produced, the required quality-assurance points, and the milestones used to indicate progress.

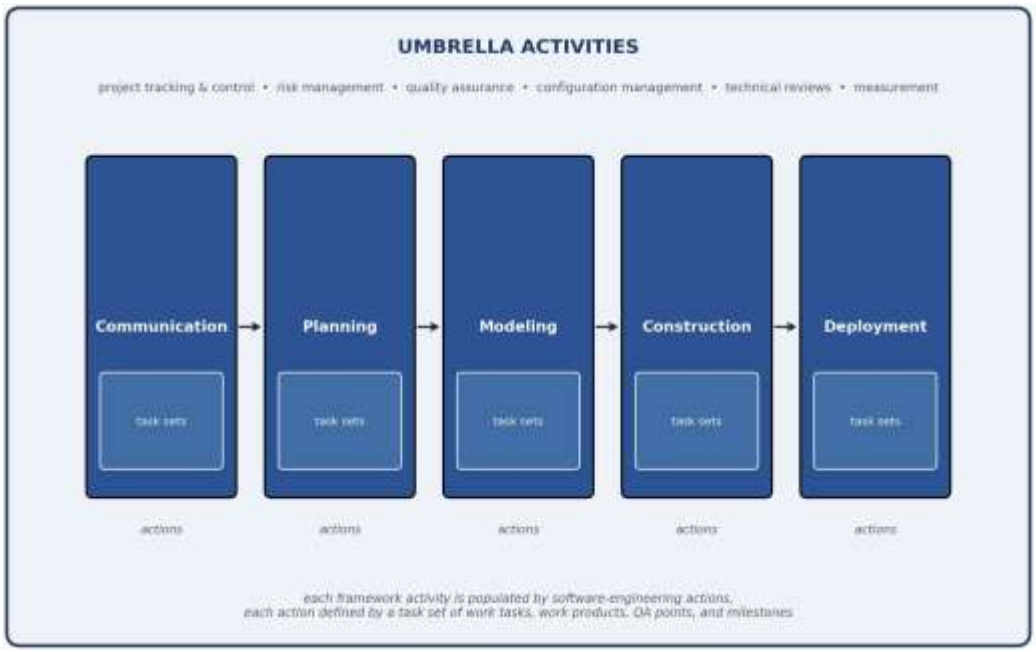


Fig 2.1 — A software process framework: umbrella activities wrap the framework activities, each populated by software engineering actions and task sets.

The generic process framework defines five framework activities — communication, planning, modeling, construction, and deployment — complemented by umbrella activities such as project tracking and control, risk management, quality assurance, configuration management, and technical reviews, applied throughout the process (see Section 1.4 above for full detail on each).

Process Flow

Process flow describes how the framework activities, and the actions/tasks within each, are organized with respect to sequence and time.

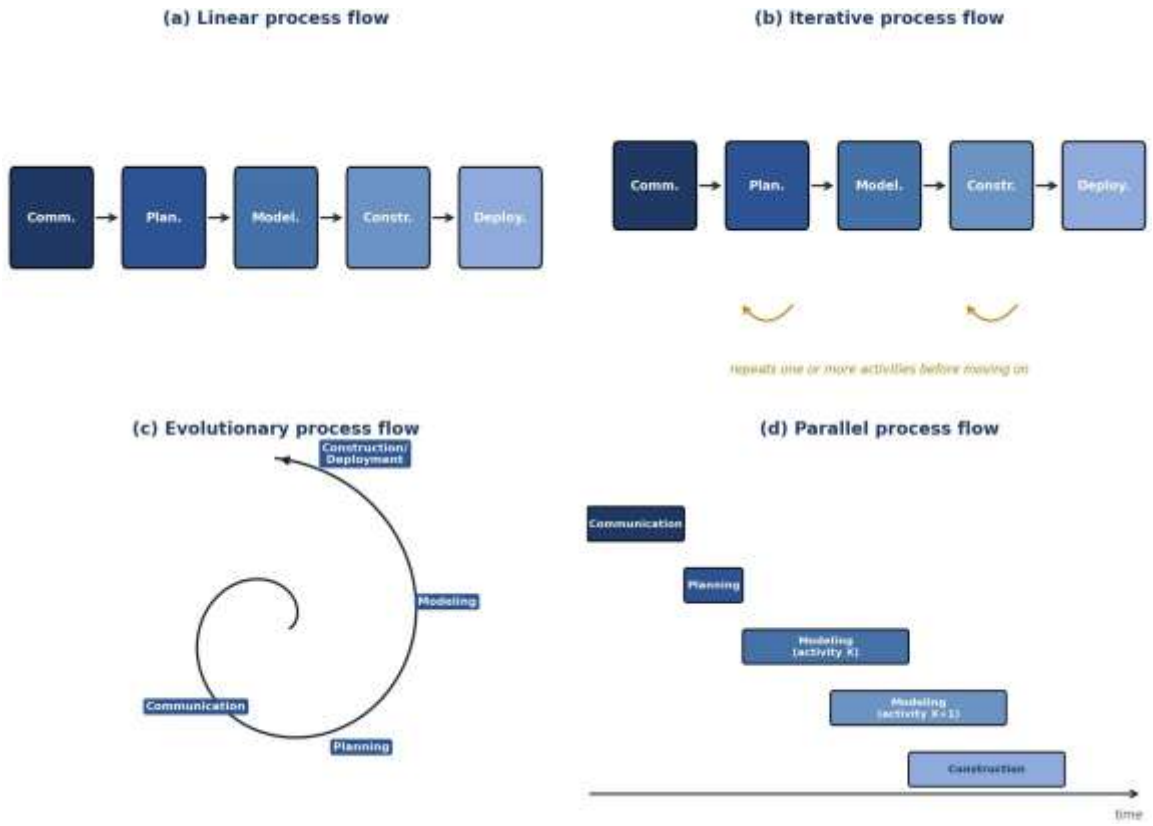


Fig 2.2 — Process flow: (a) linear — activities executed in strict sequence; (b) iterative — one or more activities repeated before moving on; (c) evolutionary — activities executed in a circular manner, each circuit producing a more complete version of the software; (d) parallel — one or more activities executed in parallel with others.

- **Linear process flow —** executes each of the five activities in strict sequence, beginning with communication and ending with deployment.
- **Iterative process flow —** repeats one or more activities before proceeding to the next.
- **Evolutionary process flow —** executes the activities in a "circular" manner, with each circuit producing a more complete version of the software.
- **Parallel process flow —** executes one or more activities in parallel with other activities (e.g., modeling one part of the software while constructing another).

2.1.1 Defining a Framework Activity

A framework activity's actions depend on the nature of the problem, the characteristics of the people doing the work, and the project's stakeholders. For a small project, communication may simply mean: (1) make contact with the stakeholder by phone, (2) discuss requirements and take notes, (3) organize notes into a brief written statement of requirements, (4) e-mail it to the stakeholder for review and approval. A larger, more

complex project with many stakeholders might expand communication into six distinct actions: inception, elicitation, elaboration, negotiation, specification, and validation — each with its own work tasks and work products.

2.1.2 Identifying a Task Set

Each software engineering action can be represented by a number of different task sets, each a collection of software engineering work tasks, related work products, quality-assurance points, and project milestones. The task set that best accommodates the needs of the project and the characteristics of the team is chosen — so an action can be adapted to the specific needs of the project and its team.

2.1.3 Process Patterns

A process pattern describes a process-related problem encountered during software engineering work, identifies the environment in which it occurs, and suggests one or more proven solutions — a template for describing problem

solutions within the context of the software process. Patterns can describe a complete process model (e.g., prototyping), a framework activity (e.g., planning), or an action within an activity (e.g., project estimating).

Ambler's template for a process pattern has nine parts: Pattern Name; Forces (the environment and issues that affect the solution); Type — Stage pattern (a framework activity, incorporating several task patterns, e.g. Establishing Communication), Task pattern (a single action/work task, e.g. Requirements Gathering), or Phase pattern (a sequence of framework activities even when the overall flow is iterative, e.g. the Spiral Model or Prototyping); Initial Context; Problem; Solution; Resulting Context; Related Patterns; and Known Uses and Examples.

Process patterns provide an effective mechanism for addressing process-related problems, letting you build a hierarchical process description from a high-level phase pattern down to stage patterns describing framework activities
— and once developed, they can be reused to define process variants.

2.2 Process Assessment and Improvement

The existence of a software process is no guarantee that software will be delivered on time or meet the customer's needs
— process patterns must be coupled with solid software engineering practice. Assessment attempts to understand the current state of the software process with the intent of improving it. Several approaches have been proposed over the years:

- **SCAMPI (Standard CMMI Assessment Method for Process Improvement) —** a five-phase assessment model (initiating, diagnosing, establishing, acting, learning) that uses the SEI CMMI as its basis.
- **CBA IPI (CMM-Based Appraisal for Internal Process Improvement) —** a diagnostic technique for assessing the relative maturity of a software organization, using the SEI CMM as its basis.
- **SPICE (ISO/IEC 15504) —** a standard defining requirements for software process assessment, helping organizations develop an objective evaluation of the efficacy of any defined process.
- **ISO 9001:2000 for Software —** a generic standard applicable to any organization wanting to improve the overall quality of its products, systems, or services — directly applicable to software organizations.

These form an SPI (Software Process Improvement) framework: the software process is examined by assessment, which identifies capabilities, strengths, weaknesses, and maturity, and leads to an improvement strategy and capability determination — the improvement strategy in turn suggests changes back to the software process, closing the loop.

2.3 Prescriptive Process Models

Prescriptive process models were originally proposed to bring order to the chaos of software development. "Prescriptive" means the model prescribes a set of process elements — framework activities, software engineering actions, tasks, work products, quality assurance, and change-control mechanisms — for every project. The "edge of chaos" is a natural, unstable, partially structured state between order and chaos; the prescriptive approach favors order and project consistency over that instability. Every process model also prescribes a process flow (work flow) describing how its elements interrelate — all models accommodate the generic framework activities, but each applies a different emphasis and invokes each activity differently.

2.3.1 The Waterfall Model (Classic Life Cycle)

The waterfall model suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment, with each phase completing before the next begins.

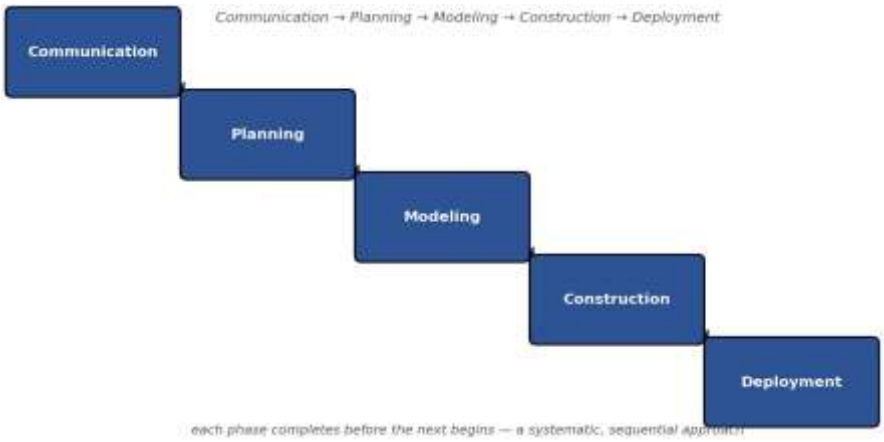


Fig 2.3 — The Waterfall Model: Communication → Planning → Modeling → Construction → Deployment.

The waterfall model is the oldest paradigm for software engineering. Problems often encountered when it is applied: real projects rarely follow the strict sequential flow proposed — a linear model accommodates iteration only indirectly, so changes cause confusion as the project proceeds; it is often difficult for the customer to state all requirements explicitly, and the model has difficulty accommodating the natural uncertainty present at the start of many projects; and the customer must have patience, since a working version of the program will not be available until late in the project time span. The model is suitable when the number of new-development efforts is limited and requirements are well defined and reasonably stable.

The V-Model

A variation in the representation of the waterfall model is the V-model, which depicts the relationship of quality-assurance (verification and validation) actions to the actions associated with communication, modeling, and early construction activities.

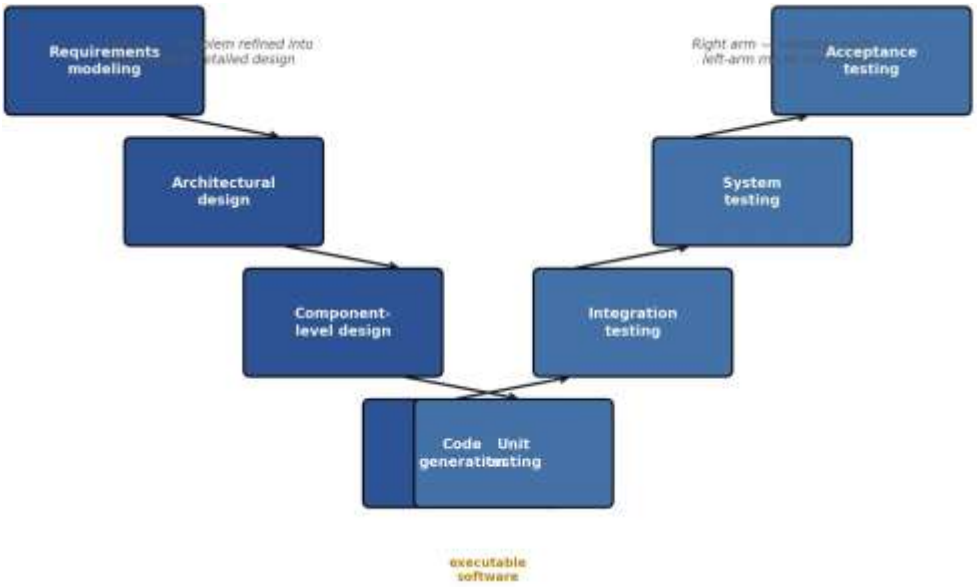


Fig 2.4 — The V-Model: left arm = requirements modeling → architectural design → component design → code generation; right arm = unit testing → integration testing → system testing → acceptance testing, validating each corresponding left-arm model on the way up to executable software.

As a team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution. Once code is generated, the team moves up the right side, performing a series of tests that validate each model created while moving down the left side — this is how the V-model visualizes verification and validation being applied to earlier engineering work.

2.3.2 Incremental Process Models

The incremental model delivers a series of releases called increments, which provide progressively more functionality for the customer as each is delivered. It combines elements of linear and parallel process flows: each linear sequence produces a deliverable increment of the software, in a manner similar to the increments produced by an evolutionary process flow. For example, word-processing software might deliver basic file management, editing, and document-production functions in increment 1; more sophisticated editing/production in increment 2; spelling and grammar checking in increment 3; and advanced page layout in increment 4.

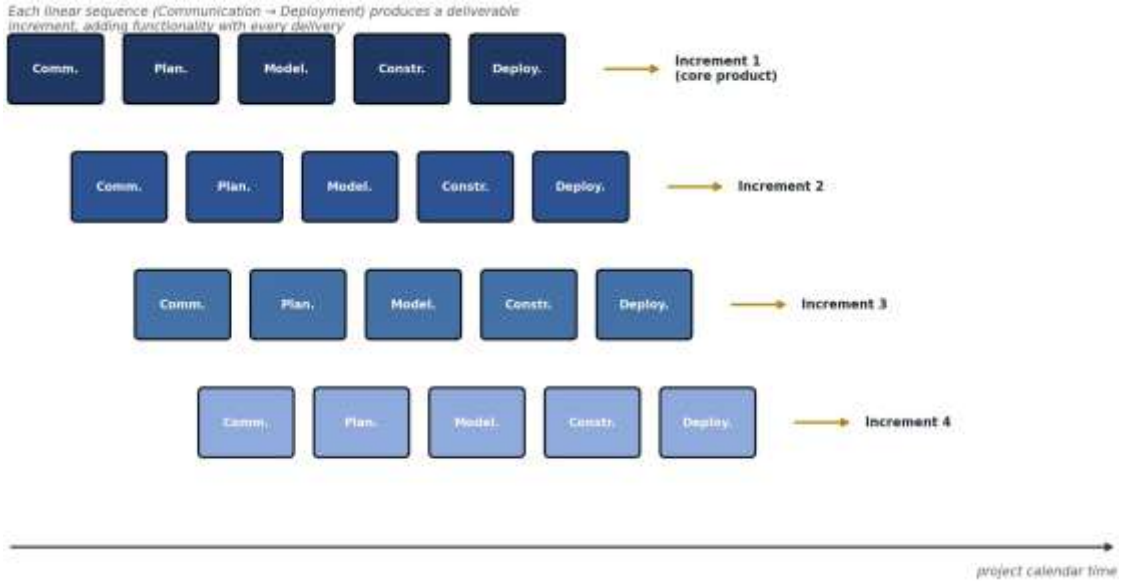


Fig 2.5 — The Incremental Model: successive increments (each passing through communication → planning → modeling → construction → deployment) are delivered in a staggered fashion as project calendar time progresses, adding software functionality and features with every delivery.

When the incremental model is used, the first increment is often a core product — basic requirements are addressed, but many extra features remain undelivered. The customer uses (and/or evaluates) the core product, and a plan is developed for the next increment, addressing modification of the core product and delivery of additional features; this repeats until the complete product is produced. Incremental development is particularly useful when staffing is unavailable for a complete implementation by an established business deadline — early increments can be implemented with fewer people, with staff added later if the core product is well received — and increments can also be planned specifically to manage technical risk.

2.3.3 Evolutionary Process Models

Evolutionary models are iterative, letting you develop increasingly more complete versions of the software with each iteration. There are two common evolutionary process models: Prototyping and the Spiral Model.

- **Prototyping Model —** used when a customer defines general objectives but not detailed requirements, or when the developer is unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form human-machine interaction should take. Prototyping begins with communication to define overall

objectives and known requirements and outline areas needing further definition; a prototyping iteration is planned quickly, and modeling occurs as a "quick design" focused on the aspects visible to end users. The quick design leads to construction of a prototype, which is deployed and evaluated by stakeholders, whose feedback refines requirements — iterating until the prototype satisfies stakeholder needs. The prototype can serve as "the first system." It can be problematic because stakeholders may see what looks like a working system without realizing it was rushed together without regard for overall quality or long-term maintainability, and the engineer may make implementation compromises (an inappropriate OS/language, an inefficient algorithm) simply to get it working quickly. Despite this, prototyping can be an effective paradigm for software engineering.

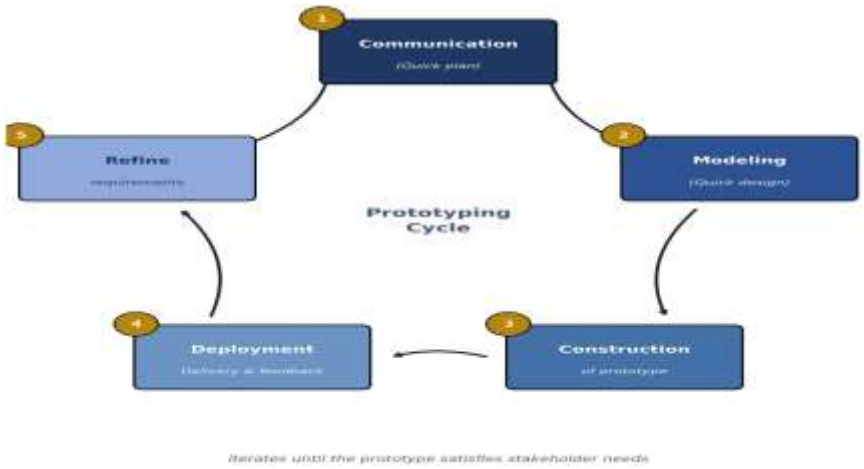


Fig 2.6 — The Prototyping Model: Communication (quick plan) → Modeling (quick design) → Construction of prototype → Deployment, delivery & feedback, iterating and refining requirements until the prototype satisfies stakeholder needs.

- **The Spiral Model (Boehm) —** originally proposed by Barry Boehm, an evolutionary software process model that couples the iterative nature of prototyping with the controlled, systematic aspects of the waterfall model, providing potential for rapid development of increasingly complete software versions. It is a risk-driven process model generator used to guide multi-stakeholder concurrent engineering of software-intensive systems, with two distinguishing features: a cyclic approach for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk, and a set of anchor point milestones for ensuring stakeholder commitment to feasible, mutually satisfactory system solutions.

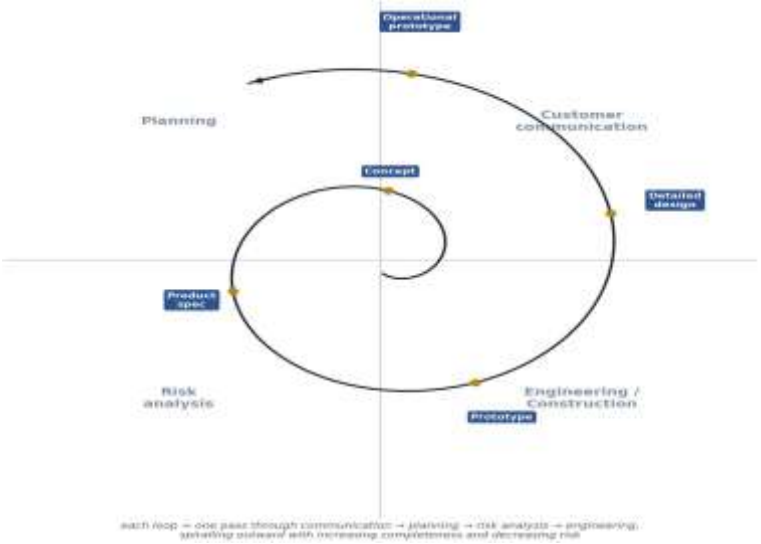


Fig 2.7 — A typical Spiral Model: each loop passes through Communication → Planning → Modeling → Construction (and Deployment), starting at the center and spiraling outward with increasing completeness, evaluating risk at every revolution.

A spiral model is divided into a set of framework activities defined by the software engineering team. As the process begins, the team performs activities implied by a clockwise circuit around the spiral, starting at the Centre. Risk is considered at each revolution, and anchor-point milestones — a combination of work products and conditions attained along the spiral's path — are noted for each evolutionary pass. The first circuit might result in a product specification; subsequent passes might produce a prototype, then progressively more sophisticated versions of the software; each pass through the planning region results in adjustments to the project plan. The spiral model can be adapted to apply throughout the life of the software — the first circuit might represent a "concept development project" continuing for multiple iterations until concept development is complete, and a later circuit might represent a "product enhancement project." It is a realistic approach for large-scale systems: because software evolves as the process progresses, both developer and customer better understand and react to risk at each level, while maintaining the systematic, stepwise approach of the classic life cycle within a more realistic, iterative framework.

2.3.4 Concurrent Models

The concurrent development model, sometimes called concurrent engineering, is represented schematically as a series of framework activities, actions, tasks, and their associated states/rules, rather than as a single sequence.

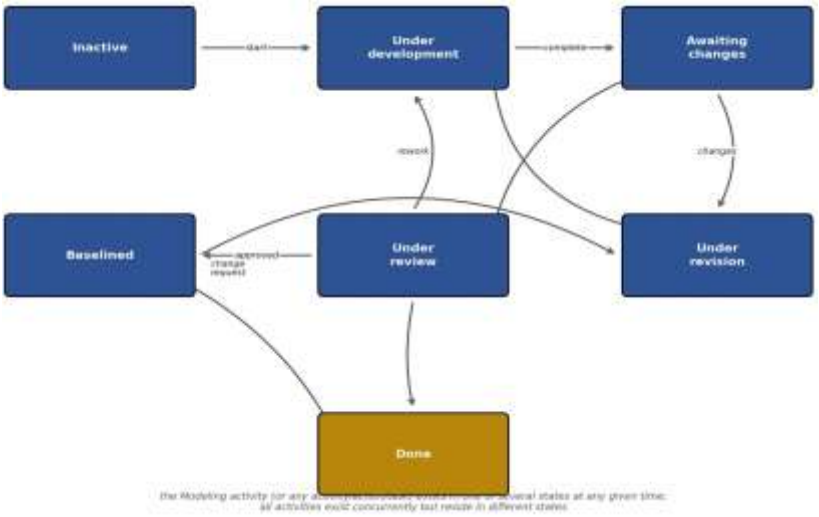


Fig 2.8 — One element of the concurrent process model: the Modeling activity may exist in any of several states (Inactive, Under development, Awaiting changes, Under revision, Under review, Baselined, Done) at any given time.

The activity "modeling" may be in any one of these states at a given time; other activities/actions/tasks (e.g., Communication, Construction) can be represented in an analogous manner. All software engineering activities exist concurrently but reside in different states — e.g., early in a project the communication activity may complete its first iteration and enter the "awaiting changes" state, while the modeling activity (previously inactive) transitions into "under development." If the customer then indicates changes are needed, the modeling activity moves from "under development" back to "awaiting changes." Concurrent modeling defines a series of events that trigger transitions from state to state for each activity, action, or task; it is applicable to all types of software development and provides an accurate picture of the current state of a project.

2.3.5 Specialized Process Models

- **Component-Based Development —** incorporates many characteristics of the spiral model — it is evolutionary in nature and demands an iterative approach — but constructs applications from prepackaged software components (conventional modules, or object-oriented classes/packages of classes). It involves: researching and evaluating available component-based products for the application domain; considering component-integration issues; designing a software architecture to accommodate the components; integrating the components into the architecture; and conducting comprehensive testing to ensure proper functionality. This model leads to software reuse, which provides engineers with a number of measurable benefits.

- **The Formal Methods Model —** encompasses a set of activities leading to formal mathematical specification of computer software, enabling you to specify, develop, and verify a computer-based system by applying rigorous mathematical notation (a variation is called clean-room software engineering). Used during development, formal methods help eliminate problems such as ambiguity, incompleteness, and inconsistency that are hard to overcome with other paradigms, discovered/corrected through mathematical analysis; used during design, they serve as a basis for program verification, offering the promise of defect-free software. Drawbacks: developing formal models is currently time-consuming and expensive; few developers have the necessary background, so extensive training is required; and the models are difficult to use as a communication mechanism with technically unsophisticated customers.
- **Aspect-Oriented Software Development (AOSD) —** also called aspect-oriented programming (AOP), defines "aspects" that express customer concerns cutting across multiple system functions, features, and information — referred to as crosscutting concerns; aspectual requirements define the crosscutting concerns that impact the software architecture. AOSD provides a process and methodological approach for defining, specifying, designing,

and constructing aspects. In the related idea of aspect-oriented component engineering (AOCE), "aspects" are horizontal slices through vertically decomposed software components, used to characterize crosscutting functional and non-functional properties of those components.

2.4 The Unified Process (UP)

Fig 2.9 — The Unified Process: the Inception, Elaboration, Construction, Transition, and Production phases spiral through the generic communication, planning, modeling, construction, and deployment activities, releasing a software increment at the end of each pass.

The Unified Process (UP) is a use-case-driven, architecture-centric, iterative and incremental software process, closely aligned with the Unified Modeling Language (UML) and developed by the originators of UML (Jacobson, Booch, and Rumbaugh) as a natural culmination of years of research into object-oriented technology, tools, and process best practice. Like other evolutionary models, the UP defines phases that occur over a series of iterations, with each iteration producing an engineering increment (a usable software release).

- **Inception —** encompasses both customer communication and planning activities: the scope of the project is identified, basic business requirements are established at a high level, a rough architecture for the system is proposed, and a plan for the iterative, incremental nature of the ensuing project is developed, along with identification of primary risks.
- **Elaboration —** encompasses the communication and modeling activities of the generic process model: it refines and expands the preliminary use cases developed in inception, and expands the architectural representation to include five different views of the software — the use-case model, the requirements model, the design model, the implementation model, and the deployment model. In some cases elaboration creates an "executable architecture baseline" representing a first cut executable system.
- **Construction —** identical to the construction activity defined for the generic software process: uses the architectural model as input and develops or acquires the software components that make each use case operational for end users. Analysis and design of remaining use cases/components are completed, and the resulting implementation is tested — culminating in the first operational software increment (an early version of the deployable product).
- **Transition —** encompasses the latter stages of construction and the first part of the generic deployment activity: software is delivered to end users for beta testing and feedback reporting defects and required changes; at the end of transition, the software increment becomes a usable software release.
- **Production —** corresponds to the deployment activity of the generic process: the

ongoing use of the software is monitored, support for the operating environment is provided, and defect reports/change requests are submitted and evaluated.

A UP project may pass through several iterations across these five phases, and each iteration produces work products such as a vision document, use-case model, software architecture description, an executable prototype/increment, a design model, source code, test plans and cases, and user manuals.

2.5 Personal and Team Process Models

Personal Software Process (PSP)

Proposed by Watts Humphrey, the Personal Software Process emphasizes personal measurement — of both the work product produced and the quality with which it is produced — so that an individual software engineer can improve his or her own process. PSP defines five framework activities:

- Planning — requirements are isolated, and based on these, estimates of the size and required resources are developed; base measures (e.g., estimated lines of code) and a total-time estimate are recorded.
- High-level design — external specifications for required components are developed and a component design is created.
- High-level design review — formal verification methods are applied to uncover errors in the design; measures are maintained for all important tasks and work results.
- Development — the component-level design is refined and reviewed, code is generated, reviewed, compiled, and tested, and measures are maintained throughout.
- Postmortem — using the measures and metrics collected, the effectiveness of the process is determined; measures/metrics should provide guidance for modifying (i.e., improving) the process for the next round.

PSP stresses the need to record and analyze the types of errors made, so the individual engineer can develop strategies to eliminate them.

Team Software Process (TSP)

Also proposed by Watts Humphrey, the goal of the Team Software Process is to build a self-directed project team that organizes itself to produce high-quality software. TSP defines the team's structure and management framework, and provides guidance

on how to establish self-directed teams that plan and track their own work, set goals, and adapt the process to their needs. Objectives of TSP include:

- Building self-directed teams that plan and track their own work, and establish goals owned by the team.
- Showing managers how to coach and motivate teams and how to help them sustain peak performance.
- Accelerating software process improvement by making CMM Level 5 behavior normal, expected practice.
- Providing improvement guidance to high-maturity organizations.
- Facilitating university teaching of industrial-grade team-skill practices consistent with the PSP each team member follows.

Each team member's work is guided by the discipline of the PSP, while TSP provides the team-level structure, roles (such as team leader, development manager, planning manager, quality/process manager, and support manager), and coordination needed to integrate individually PSP-trained engineers into an effective, self-managed team.